

LUX TECHNICAL NOTE

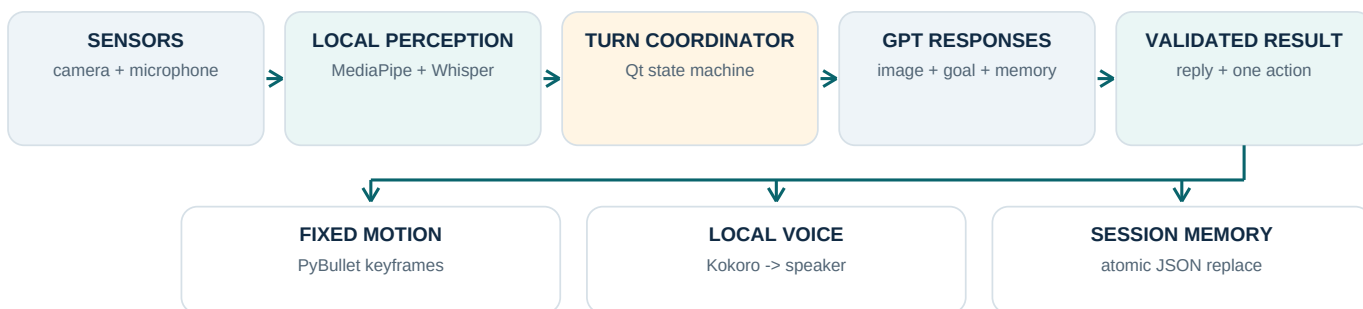
CPU-first expressive lamp character | Hybrid local speech + cloud vision-language



System at a glance

Lux is a continuous desktop character built around the supplied five-DOF lamp URDF. It combines local engagement, speech recognition, speech synthesis, simulation, and audio with a cloud vision-language planner. The design keeps the generative model above a strict application-owned action boundary.

ARCHITECTURE AND DATA FLOW



All model outputs cross a schema and whitelist boundary before execution or storage.

Turn protocol

01 Engage

MediaPipe combines head pose, iris offset, eye openness, and face size. A 0.7 s enter / 3.0 s exit hysteresis prevents flicker.

02 Listen

An adaptive energy endpoint detector submits after 2.0 s of continuous silence. Audio stays in memory.

03 Understand

Faster-Whisper Small transcribes locally on CPU; the current frame, transcript, and complete session JSON form the GPT request.

04 Act and observe

GPT returns one whitelisted action. Lux executes fixed keyframes; if another view is requested, it captures a newer frame and calls GPT again.

05 Answer and commit

Only the final validated decision is spoken. Kokoro renders voice locally, then memory is atomically replaced and listening reopens after playback.

Model-to-action contract

STRUCTURED DECISION

reply | motion_labels (0 or 1) | request_another_observation | updated_memory

Allowed actions: inspect_left, inspect_right, nod_yes, shake_no. The model never emits joint angles. At most three camera observations are permitted per user turn; planning replies are neither spoken nor committed.

Character behavior across a turn



Turn lock keeps Think/Answer active through every GPT wait and the full voice playback; listening audio remains suspended.

IMPLEMENTATION, EVIDENCE & LIMITS

Measured on the development laptop; target-hardware gaps are called out explicitly.

Implementation choices

- Perception: MediaPipe Face Landmarker runs locally at 640 x 480. The signals are interaction heuristics, not a claim about cognitive attention.
- Speech: Faster-Whisper Small uses CPU int8, beam size 5, and VAD. Kokoro-82M (af_heart, 24 kHz) runs locally and is warmed at startup.
- Control: PyBullet loads the fixed-base URDF and CPU TinyRenderer. Eleven five-joint keyframe behaviors are checked against URDF hard limits.
- Concurrency: camera, microphone, GPT, TTS, and character audio use Qt workers; the GUI thread owns turn state and memory commit.
- Memory: compact conversation and scene summaries are schema-validated and atomically stored for the current run; startup resets the file.

Deployment and data

- Target: Ubuntu 24.04, four CPU cores, 8 GB RAM, no CUDA. setup.sh creates Python 3.11, installs native libraries, downloads models, and runs tests.
- Local only: raw microphone audio, engagement inference, Whisper, Kokoro, motion, light, SFX, and music.
- Sent to OpenAI: transcript, low-detail camera frame(s), and session JSON. Requests set store=False; GPT is the only billable runtime component.

Evidence and measurements

LIVE ENGAGEMENT

60 / 60

30 look-toward and 30 turn-away trials all produced the expected engagement transition on the real camera.

WARM STT / TTS

4.7 s / 2.6 s

Mean latency across 30 speech-to-text trials and 30 text-to-speech trials with models resident in memory.

GPT RESPONSE LATENCY

8.2 s / 2.7 s

Mean across 30 first requests and 30 post-first requests. Network and remote-service conditions affect these values.

COLD LOCAL LATENCY

30.0 s

Mean across 30 cold starts with model weights cached. Startup preloading moves this cost before the first turn.

FULL-APP MEMORY

2.01 GiB peak

90 s process-tree profile: 1.56 GiB steady mean (45-90 s). All local subsystems were enabled.

FULL-APP CPU

1.68 avg / 5.35 peak

45-90 s steady mean and 90 s peak. Host: Ryzen 9 8945HX (16C/32T), Windows 11, 31.8 GiB RAM.

Tradeoffs and known limitations

- GPT vision-language reasoning is flexible but needs Wi-Fi, an API key, incurs cost, and adds variable latency.
- Fixed semantic actions are inspectable and safe, but less expressive than continuous learned control.
- The base is fixed; there is no real-robot transport, collision planning, torque control, calibration, or emergency stop.
- Engagement uses one face and hand-tuned thresholds; lighting, glasses, camera placement, and face geometry can reduce reliability.
- A turn is capped at three visual observations and one action per model response, limiting long plans.
- Ubuntu package names and script syntax are checked, but camera, audio, GUI, latency, and memory still require physical Ubuntu validation.

Completion boundary

COMPLETED

Live desktop loop; local STT/TTS; GPT scene reasoning; session memory; iterative observation; simulated motion, light, SFX, music; 67 automated tests.

INTENTIONALLY LEFT OUT

Real hardware transport and safety stack; continuous VLA joint control; cross-session identity; statistical user study; target-laptop hardware certification.

MEASUREMENT SCOPE

Latency values are arithmetic means from 30 trials per condition. Full-app CPU and memory use one 90 s process-tree profile with GUI, camera, MediaPipe, PyBullet, audio, Whisper, and Kokoro enabled; no GPT turn was issued.